

코딩 테스트 대비 알고리즘 스터디

6회차: DP 알고리즘 (3)

14501: 퇴사

문제

상담원으로 일하고 있는 백준이는 퇴사를 하려고 한다.

오늘부터 $N+1$ 일째 되는 날 퇴사를 하기 위해서, 남은 N 일 동안 최대한 많은 상담을 하려고 한다.

백준이는 비서에게 최대한 많은 상담을 잡으라고 부탁을 했고, 비서는 하루에 하나씩 서로 다른 사람의 상담을 잡아놓았다.

각각의 상담은 상담을 완료하는데 걸리는 시간 T_i 와 상담을 했을 때 받을 수 있는 금액 P_i 로 이루어져 있다.

$N = 7$ 인 경우에 다음과 같은 상담 일정표를 보자.

	1일	2일	3일	4일	5일	6일	7일
T_i	3	5	1	1	2	4	2
P_i	10	20	10	20	15	40	200

상담을 적절히 했을 때, 백준이가 얻을 수 있는 최대 수익을 구하는 프로그램을 작성하시오.

입출력 예제

예제 입력 1 복사

```
7
3 10
5 20
1 10
1 20
2 15
4 40
2 200
```

예제 출력 1 복사

```
45
```

예제 입력 2 복사

```
10
1 1
1 2
1 3
1 4
1 5
1 6
1 7
1 8
1 9
1 10
```

예제 출력 2 복사

```
55
```

문제 접근법

기간
비용

	1일	2일	3일	4일	5일	6일	7일
T_i	3	5	1	1	2	4	2
P_i	10	20	10	20	15	40	200

상담 중

상담 결정

상담 중

더 높은 비용의 상담이
있으면 손해

상담 중

상담 기간과 비용 간의 trade-off를
고려하여 상담 일정安排할 필요 有

→ 고급 알고리즘이 있을 것 같지만,
가장 단순한 건 Brute-Force

Brute-Force적 접근

	1일	2일	3일	4일	5일	6일	7일
T_i	3	5	1	1	2	4	2
P_i	10	20	10	20	15	40	200

(1) DFS로 모든 경우의 수를 순회 (모든 날짜에 대해 상담을 진행 할 수도 & 건너뛴 수도)

(2) 모든 경우의 수에 대한 총 상담 비용을 계산

(3) 가장 많은 수익을 올린 경우가 정답

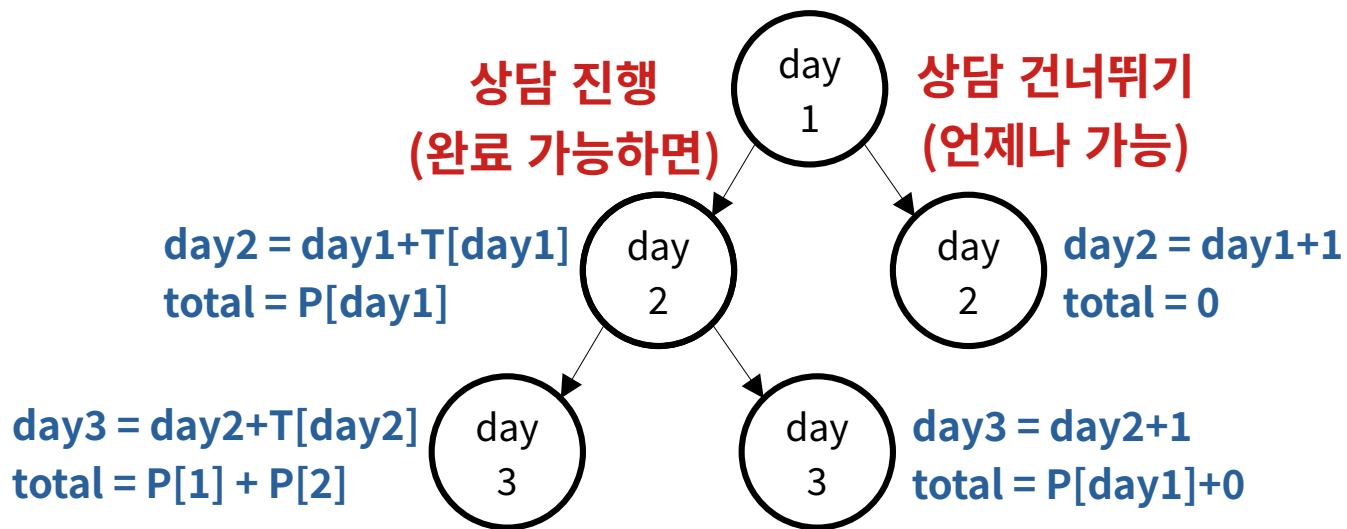
입력 조건

첫째 줄에 N ($1 \leq N \leq 15$)이 주어진다.

최악의 경우 $\rightarrow 2^{15} = 32768$

Brute-Force로 풀어도 시간 충분

DFS 구조 도식화



종료조건 → 'day'가 퇴사 일이 되면 더 이상 상담 불가능
현재 총 상담 비용을 기존 최대 비용과 비교 → 최댓값 갱신

```
N = int(input())

T = [0] * N
P = [0] * N

for i in range(N):
    T[i], P[i] = map(int, input().split())
```

```
def find_max_profit(curr_day, total_price):
    global maximum

    # 종료 조건 (퇴사일 도달)
    if curr_day >= N:
        maximum = max(maximum, total_price)

    # 아직 상담을 더 진행할 수 있는 경우
    else:
        # 상담 진행 선택 (퇴사 전에 완료 가능할 때)
        if curr_day + T[curr_day] <= N:
            find_max_profit(curr_day + T[curr_day], total_price + P[curr_day])
        # 상담 건너 뛰기 (언제나 가능)
        find_max_profit(curr_day + 1, total_price)
```

```
maximum = 0
find_max_profit(0, 0)

print(maximum)
```

문제 접근법 (DP)

DP 문제를 풀 때, 거꾸로 접근하는 방식이 유효할 때가 많다.

	1일	2일	3일	4일	5일	6일	7일	초기화
T_i	3	5	1	1	2	4	2	총 비용: 0
P_i	10	20	10	20	15	40	200	

↓
**회사 전까지
 상담 불가능**

날짜	1	2	3	4	5	6	7	초기화
DP 배열	10+35 45	20+0 45	45 35	35 15	15 0	0 0	0 0	0


```
dp = [0] * (N+1)
for day in range(N-1, -1, -1):
    # 아래 두 가지 중 최대값 선택

    # (1) 현재 날짜(day)에 상담이 완료 가능한 경우
    consult = 0
    if day + T[day] <= N:
        consult = dp[day+T[day]] + P[day]
    # (2) 상담 건너 뛰는 경우
    skip = dp[day+1]
    dp[day] = max(consult, skip)

print(dp[0])
```